

WEEK 4 • SELF-STUDY MODULE

The Linux File System and Storage Management

IT 4112 — System Administration and Maintenance

How to use this module

This module replaces one face-to-face session. Work through it on your own, at your own pace, with your Ubuntu Server virtual machine open beside you. It is written to be read in order — each section builds on the one before it.

Element	What it means
TRY THIS	Open your VM and run the commands. Reading about a command is not the same as running it.
PAUSE AND THINK	Stop and answer the question before reading on. The answer is in the paragraph that follows.
WATCH OUT	A mistake that costs people real servers. Read these twice.
WORTH KNOWING	Background that explains why something works the way it does.

At a glance	Details
Estimated study time	2 hours, including the hands-on parts
What you need	Your Ubuntu Server VM from PT1 and PT2, and about 4 GB of free disk on the host
Assessment	The self-check quiz at the end, and Performance Task 3
Prepares you for	PT3 — provisioning, partitioning and mounting storage

What you should be able to do afterwards

- Explain how the Linux file system is organised and why it has a single root instead of drive letters.
- Name the purpose of the main system directories and say which ones must never be touched.
- Explain what it means that in Linux, everything is a file — including hardware.
- Tell the difference between a disk, a partition, a file system and a mount point.
- Choose between ext4 and NTFS for a given purpose and defend the choice.
- Use lsblk, fdisk, mkfs, mount, umount and df with confidence.
- Add a virtual hard disk to a VirtualBox guest.
- Explain why unmounting matters, and what goes wrong when you skip it.

1 Two Things That Went Wrong

Before any commands, two stories. Both are ordinary. Both cost somebody something. Everything else in this module exists to stop them happening to you.

1.1 The case of the disappearing thesis

Linda is a university student who dual-boots Ubuntu and Windows on her laptop. She is finishing a fifty-page thesis — months of work.

She saves it in Ubuntu as `/home/linda/MyCapstone.docx`. Later she reboots into Windows to show her professor, opens her USB drive, searches for the file — and it is not there.

She panics, reboots back into Ubuntu, and there it is, safe and sound. Relieved, she tries to copy it to her USB:

```
linda@ubuntu: ~  
cp mycapstone.docx /media/usb/  
cp: cannot stat 'mycapstone.docx': No such file or directory
```

She looks again. The file is called `MyCapstone.docx`, with two capital letters. She retypes it exactly and the copy succeeds. But when she plugs the USB into a Windows lab computer, the file is missing again.

PAUSE AND THINK

Three separate things have gone wrong in Linda's story. Before you read on, write down what you think each one is. Two of them are about how Linux names and stores files. One is about how she removed the drive.

What actually went wrong

Symptom	Cause
"No such file or directory" for a filename that looked correct	Case sensitivity. Linux treats <code>MyCapstone.docx</code> and <code>mycapstone.docx</code> as two completely different files. Windows treats them as the same name, and will not let both exist.
The file was invisible on the Windows lab computer	File system compatibility. The USB had been formatted <code>ext4</code> , which is a Linux-native file system. Windows cannot read <code>ext4</code> at all, so the drive looks blank or asks to be formatted.
The file was partially written and unusable	The USB was pulled out without being unmounted. Linux had not finished writing the data, so what reached the drive was incomplete.
A fourth risk she never noticed	From Ubuntu she had write access to the Windows partition. One careless delete in the wrong folder and Windows would not have booted at all.

1.2 The intern who broke the server

Vince is a new IT intern at a small company. He is asked to free up space on the company's Ubuntu server, because a critical application is failing with "disk full" errors.

He logs in, opens the file manager, and sees that `/home` holds a lot of old user files. But the error was on the root partition (`/`), not on `/home` — and he does not check. Instead of measuring, he guesses.

He browses to `/bin`. He sees files named `init`, `mount`, `cp`. He thinks: these look like old installer files, I will move them to a backup folder. He copies several of them to his home directory. Then he deletes the originals to save space.



Deleting things you do not understand is the fastest way to break a server.

A few minutes later the entire server crashes. Nobody can log in. The website is down. The database is unreachable. IT staff discover the missing system binaries. The server has to be rebuilt from backup — eight hours of downtime and real lost revenue.

What actually went wrong

Mistake	What he should have known
He misunderstood the file hierarchy	<code>/bin</code> holds essential system binaries. Without them, the operating system cannot run — not even enough to let someone log in and fix it.
He had superuser power without understanding it	Only a superuser can modify system directories. Vince had <code>sudo</code> access, but <code>sudo</code> does not come with judgement attached. It removes the safety rail; it does not tell you where the edge is.
He never diagnosed the problem	One command — <code>df -h</code> — would have shown him exactly which partition was full. He guessed instead of measuring.
He did not understand mounting and partitions	The full partition may have been a separate volume such as <code>/var/log</code> . Deleting from <code>/bin</code> would not have freed a single byte on it.

THE LESSON

In Linux the file system is not just folders. It is the foundation the whole operating system stands on.

Delete the wrong file and the system will not boot. Edit `/etc/passwd` carelessly and nobody can log in. Forget to unmount a USB and you risk corrupting data. Mount a Windows partition and delete from it and Windows will not start.

Understanding the file system is not optional. It is the difference between an administrator and a liability.

The concepts these two stories introduce

Concept	Why it mattered
Superuser power	Can break the system beyond repair. Always log in as a normal user, even when you are the administrator.
<code>/bin</code> , <code>/sbin</code> , <code>/etc</code>	Vital system directories. Nothing in them is spare.

Concept	Why it mattered
Mounting	Required to reach storage safely — and required to leave it safely.
File system hierarchy	Everything is rooted in a single /. There is no separate D: drive to be safe in.
Case sensitivity	MyFile.txt and myfile.txt are two different files.

2 Linux and Windows: Three Differences That Matter

If you have only used Windows, three habits will trip you up. They are not difficult — they are just different, and each one caused part of Linda's problem.

2.1 Case sensitivity

	Linux	Windows
Are myfile.txt and MyFile.txt the same?	No — two different files	Yes — the same file
Can both exist in one folder?	Yes	No

This is why Linda's copy failed. The shell was not being difficult; it genuinely could not find a file by that name, because that file did not exist.

2.2 Drive letters versus a single root

Windows gives every drive its own letter — C:, D:, E: — and each letter is the top of its own separate tree. Linux has exactly one tree. Its top is the forward slash, /. Every disk, partition, USB stick and network share is attached somewhere inside that one tree.

Attaching storage into the tree is called mounting, and the directory you attach it to is called a mount point. There is no D: drive to plug in. Instead you decide where in the tree the new storage appears.

2.3 Path separators

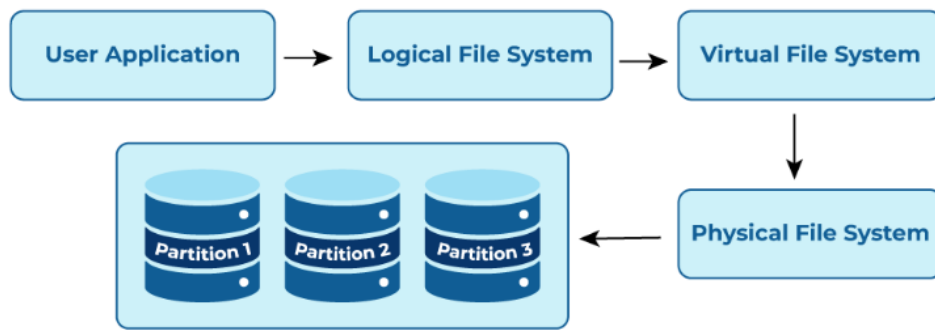
	Linux	Windows
Separator	Forward slash /	Backslash \
Example path	/home/alex/report.txt	C:\Users\Alex\report.txt

WORTH KNOWING

The idea of a path is identical in both systems. A folder is a container, inside it can be another container, and inside that is a file. Only the punctuation and the starting point differ. If you can read a Windows path, you can read a Linux one.

3 What a File System Actually Is

A file system is the organised structure that lets Linux store and retrieve data. That sounds abstract, so be specific about what it is not: it is not the folders you see. Folders are what the structure shows you. The file system is the bookkeeping underneath — the index that records which blocks on the disk hold which file, who owns it, when it changed, and which blocks are still free.



From your program down to the physical disk. Each layer hides the one below it.

Reading that diagram from the top:

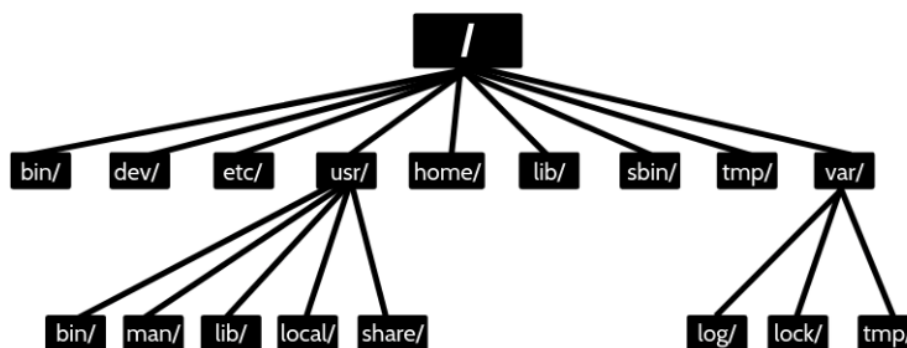
Layer	What it does
User application	Your program says "open report.txt". It knows nothing about disks.
Logical file system	Translates that name into a request the system can act on, and checks permissions.
Virtual file system	The translator that lets one set of commands work on ext4, NTFS, FAT32 and network shares alike. This is why the same cp command works on any of them.
Physical file system	Talks to the actual hardware — the partitions and the blocks inside them.

WHY THE VIRTUAL LAYER MATTERS TO YOU

In Performance Task 3 you will mount an ext4 partition and an NTFS partition side by side, and then copy files into both with the same command. That works because of the virtual file system layer. Without it you would need one set of tools per file system type.

4 The File System Tree

Every path in Linux starts from the root: a single forward slash, /. Everything else hangs beneath it.



The standard Linux directory hierarchy, all descending from a single root.

The directories you need to recognise

Directory	What lives there	Safe to touch?
/	The root of everything. Not a drive — the top of the single tree.	Structure: no
/home	User files. /home/alex, /home/linda, and so on.	Yes — your own

Directory	What lives there	Safe to touch?
/bin	Essential system binaries: ls, cp, mount, and the rest. The tools needed just to get the system running.	Never
/sbin	System binaries reserved for the administrator.	Never
/etc	Configuration files. /etc/passwd, /etc/fstab, network settings.	Only with care
/tmp	Temporary files. Usually cleared on reboot.	Yes
/media	Where the system automatically mounts removable devices — USB sticks, optical discs.	Yes
/mnt	Where an administrator mounts storage manually. This is where your PT3 mount points go.	Yes
/dev	Device files. Your disks and partitions appear here as /dev/sda, /dev/sdb1.	Read, don't edit
/proc and /sys	Virtual directories. They contain no real files — the kernel generates their contents live.	Read only
/var	Variable data: logs, mail queues, databases. /var/log is often on its own partition.	With care

THIS IS THE TABLE VINCE NEEDED

He deleted from /bin because he did not recognise the names in it. Everything in /bin is load-bearing. If you cannot say what a directory is for, that is the reason to leave it alone — not a reason to clean it out.

TRY THIS

Open your VM and look at the tree for yourself. The tree command may need installing first.

```
ls /
ls -l /home
sudo apt install tree    # if it is not already there
tree -L 1 /
```

5 Everything Is a File — Even Hardware

This is the idea that separates Linux from Windows more than any other. In Linux, hardware is represented as files. Your hard drive is a file. Your USB port is a file. Your CPU information is a file.

```
ian@ian-VirtualBox: /dev
nvrw      tty18  tty49  tty
port      tty19  tty5   tty
ppp       tty2   tty50  tty
psaux     tty20  tty51  tty
ptmx      tty21  tty52  tty
pts       tty22  tty53  tty
random    tty23  tty54  tty
rfkill    tty24  tty55  tty
rtc       tty25  tty56  tty
rtc0      tty26  tty57  tty
sda       tty27  tty58  tty
sda1      tty28  tty59  tty
sda2      tty29  tty6   tty
sda3      tty3   tty60  tty
```

A listing of /dev. Every entry here is a piece of hardware or a kernel interface.

Path	What it represents
/dev/sda	Your first hard drive, as a whole
/dev/sdb1	The first partition on the second drive
/dev/ttyUSB0	A USB-to-serial device
/proc/cpuinfo	Details about your CPU — generated live, not stored anywhere
/sys/class/backlight	Screen brightness control
/dev/null	A file that discards everything written to it

WHY THIS DESIGN IS USEFUL

Because hardware is a file, the ordinary file tools work on it. You can read CPU details with `cat`. You can copy an entire disk with `dd`, the same command you would use for a file. You can redirect unwanted output into `/dev/null`. One consistent set of tools works everywhere, which is exactly what makes Linux so scriptable.

TRY THIS

See the design for yourself:

```
cat /proc/cpuinfo | head -20
ls -l /dev/sd*
echo "this vanishes" > /dev/null
```

6 Disks, Partitions and How They Are Named

Three words that are often used loosely, and mean quite different things:

Term	Definition	Real-world comparison
Disk	The physical storage device — or, in a VM, the .vdi file pretending to be one.	A whole filing cabinet
Partition	A logical division of the disk. One disk can hold several, each independent of the others.	One drawer in that cabinet

Term	Definition	Real-world comparison
File system	The structure written into a partition so it can hold files. ext4, NTFS, FAT32.	The folder dividers inside the drawer

6.1 How Linux names disks

Disk and partition names live in /dev and follow a simple pattern.

```
ian@ian-VirtualBox:/dev$ ls sd*
sda  sda1  sda2  sda3
```

Listing /dev/sd shows the first drive and its three partitions.*

Name	Meaning
/dev/sda	First drive
/dev/sdb	Second drive
/dev/sdc	Third drive — and so on alphabetically
/dev/sda1	First partition on the first drive
/dev/sda2	Second partition on the first drive
/dev/sda5	The start of logical partitions on an older MS-DOS partition table

PAUSE AND THINK

You add two blank disks to a machine that already has one. What names will Linux most likely give them, and what will the first partition on the larger new disk be called?

Most likely /dev/sdb and /dev/sdc, and the first partition on the second new disk would be /dev/sdc1. Most likely — not certainly. Device letters are assigned in the order the kernel detects hardware, and that order can change. This is why professionals identify disks by size and UUID before running anything destructive, and it is why PT3 asks you to confirm your device names rather than assume them.

WATCH OUT

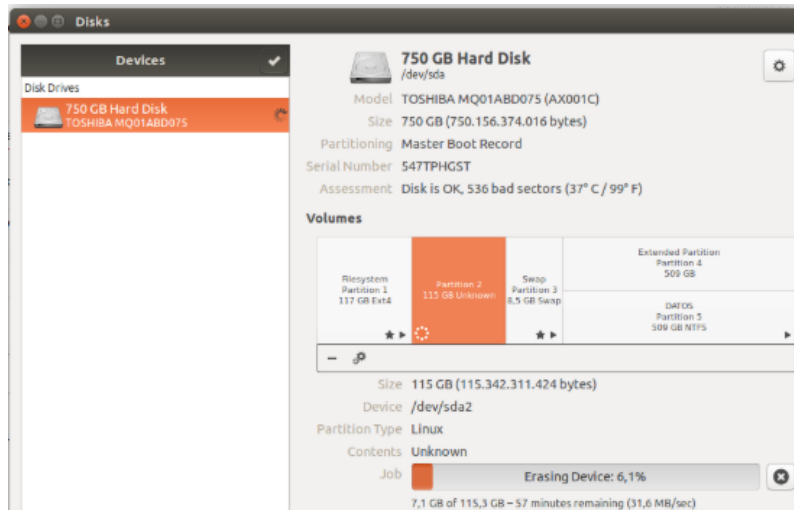
Every partitioning and formatting command names a device, and every one of them destroys what is already there. Typing sda where you meant sdb wipes the operating system you are working on. Read the device name out loud before you press Enter — this is a habit that experienced administrators actually keep.

7 Graphical Tools

On a desktop Ubuntu machine you have several graphical partitioning tools. You will not use them in the lab — Ubuntu Server has no desktop — but you should recognise them, because they are what you will meet on a workstation.

7.1 Disks (GNOME Disks)

Built into Ubuntu Desktop. It formats, mounts and unmounts volumes and reads SMART health data from the drive.

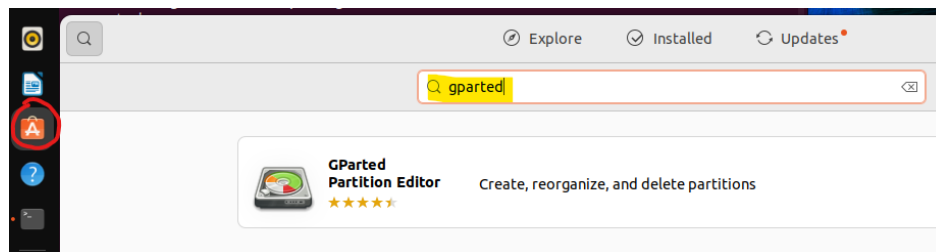


GNOME Disks showing a drive, its volumes and its health information.

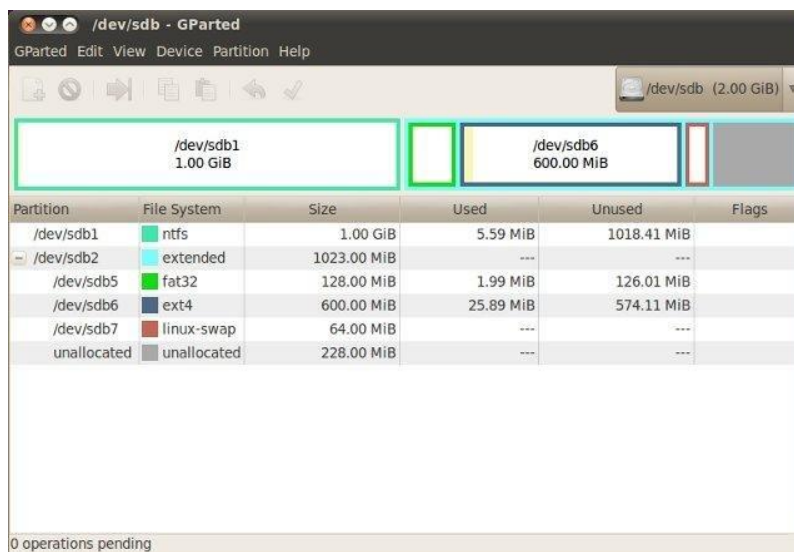
The usual sequence in Disks: open it from Applications, select a drive, create, delete or format a partition, then mount or unmount the volume.

7.2 GParted

More capable than Disks, and the tool of choice when you need to resize an existing partition without losing what is on it. It is not installed by default.



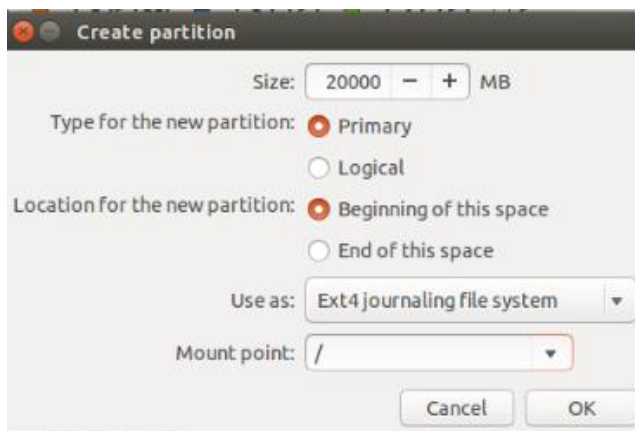
GParted in Ubuntu Software. Install it before first use.



GParted showing the partitions on a disk, with file system types and free space.

GParted queues your changes rather than applying them immediately — notice the pending-operations count at the bottom of the window. Nothing happens to the disk until you click Apply. That is the graphical equivalent of the w key in fdisk, which you will meet in the next section.

7.3 Creating a partition in a graphical tool

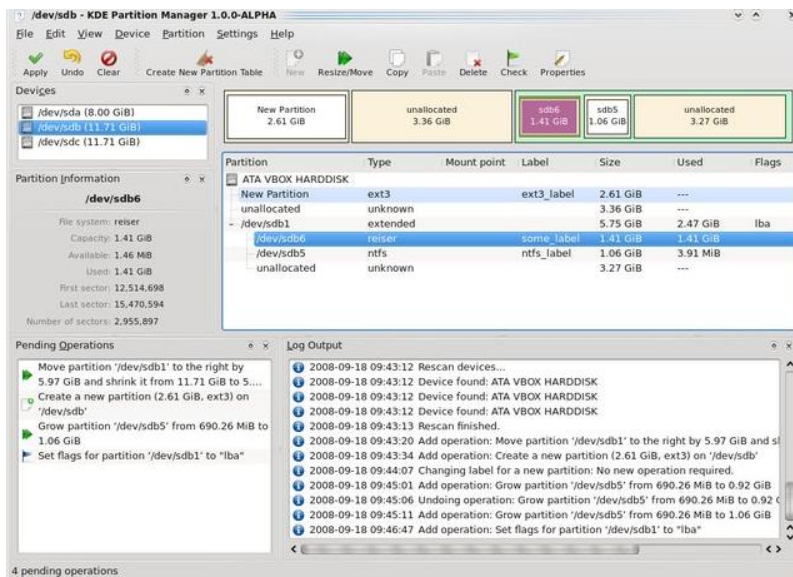


The create-partition dialog: size, primary or logical, file system, and mount point — all in one screen.

Look carefully at that dialog, because it asks for four things at once: how big the partition should be, whether it is primary or logical, which file system to put in it, and where to mount it. On the command line these are four separate operations — `fdisk`, then `mkfs`, then `mkdir`, then `mount`. The graphical tool hides the sequence. Knowing the sequence is what lets you fix things when the graphical tool is not available, which on a server is always.

7.4 Other tools you may meet

Tool	Where you find it	What it is for
Disks	Built into Ubuntu Desktop	Format, mount, check drive health
GParted	Install separately	Resize, create, delete, move partitions
Ubuntu Installer	During installation	Partitioning the disk as you install
KDE Partition Manager	On KDE-based systems	The KDE equivalent of GParted



KDE Partition Manager — a different interface, the same underlying operations.

8 The Command Line: Partition, Format, Mount

This is the section that matters most. Ubuntu Server has no graphical tools, so these commands are the only way to do the job. There are four steps, always in this order, and skipping one is the source of most of the errors you will hit in the lab.

Step	Command	What it produces
1. Partition	fdisk	A region of the disk, reserved but empty
2. Format	mkfs.ext4 / mkfs.ntfs	A file system written into that region
3. Create a mount point	mkdir	An empty directory to attach it to
4. Mount	mount	The partition, now reachable through that directory

8.1 Looking before you touch anything

```
ubuntu-server: ~
lsblk          # tree of every block device, size, and mount point
lsblk -f       # the same, plus file system type, label and UUID
sudo fdisk -l  # detailed partition table for every disk
df -h         # space used and free on every mounted file system
mount         # everything currently mounted, with its options
```

LEARN lsblk -f FIRST

Of everything above, lsblk -f is the one to internalise. In a single screen it tells you what disks exist, how they are divided, what file system each partition holds, and where each one is attached. Almost every storage question starts with it.

8.2 Partitioning with fdisk

fdisk is interactive. You start it against one disk, then type single-letter commands. Nothing is written to the disk until you press w — which means a mistake costs you nothing as long as you catch it before then.

```
ubuntu-server: ~
sudo fdisk /dev/sdb
```

Key	Action	Key	Action
m	Show the help menu	t	Change a partition's type code
g	New GPT partition table	L	List available type codes
o	New MS-DOS partition table	p	Print the current table
n	Create a new partition	w	Write changes and exit
d	Delete a partition	q	Quit WITHOUT saving

SIZE SHORTHAND

When fdisk asks for the last sector, you can type a size instead of a sector number: **+15G** for 15 gibibytes, **+500M** for 500 mebibytes, or press Enter to use all the space that is left.

Other tools do the same job. parted handles very large disks and scripted partitioning; cfdisk is an older menu-driven interface that many administrators still prefer for its clarity. fdisk is enough for everything in this course.

8.3 Formatting

A partition holds nothing until a file system is written into it. That is what mkfs does — make file system.

```
ubuntu-server: ~
sudo mkfs.ext4 /dev/sdb1      # the Linux default
sudo mkfs.vfat -F 32 /dev/sdb1 # FAT32, for maximum compatibility
sudo mkfs.ntfs -f /dev/sdb1   # NTFS, quick format
```

Choosing a file system

File system	Best for	Watch out for
ext4	Anything that stays on Linux. Journaling, full Unix permissions, mature and fast.	Windows cannot read it at all — this is what hid Linda's thesis.
NTFS	Storage that a Windows machine will also need to read.	Permissions do not map cleanly onto Linux ownership. Needs the ntfs-3g package.
FAT32	USB sticks that must work everywhere, including cameras and car stereos.	No file larger than 4 GB. No permissions at all.

mkfs IS NOT REVERSIBLE

Formatting writes a new, empty file system over whatever was there. There is no confirmation prompt on most of these commands and no undo. Check the device name twice.

8.4 Mounting and unmounting

A formatted partition still cannot be reached. Mounting attaches it to a directory in the tree. After that, anything you write inside that directory goes to the partition instead of to the system disk.

```
ubuntu-server: ~
sudo mkdir /media/usb      # create the mount point
sudo mount /dev/sdb1 /media/usb # attach the partition to it

sudo umount /media/usb     # detach it again
```

IT IS SPELLED **umount**, NOT **unmount**

There is no **n**. This trips up nearly everyone at least once, and the error message — command not found — does not make the reason obvious.

Where to mount things

Directory	Used for	Who mounts there
/media	Removable devices: USB sticks, SD cards, optical discs	The system, automatically
/mnt	Storage attached deliberately by an administrator	You, by hand

PAUSE AND THINK

What do you think happens if you mount a partition onto a directory that already has files in it?

The existing files are hidden — not deleted, just covered over, the way a sheet of paper covers whatever is underneath it. They come back the instant you unmount. This is confusing enough that mount points should always be empty directories created for the purpose.

WHY UNMOUNTING IS NOT OPTIONAL

When you write a file, Linux does not necessarily send it to the disk straight away. It holds the data in memory and writes it out when convenient, because that is much faster. Unmounting forces every pending write to complete, then detaches the file system.

Pull the drive before that happens and you get exactly what Linda got: a file that is half-written and useless. The directory to be unmounted must also be inactive — nothing may be using it, including your own shell sitting inside it.

When unmount refuses

Message	Meaning	Fix
target is busy	Something is using the file system — usually your own shell	cd / then try again; use lsof +D /path to identify the process
not mounted	It is already detached	Nothing to do — confirm with lsblk
not found	You typed unmount instead of umount	Drop the n

9 Working With Files Once the Disk Is Ready

Once storage is mounted, these are the commands you use every day. You met most of them in PT1; they are collected here because you will need them inside your new partitions.

Moving around and looking

Command	What it does
ls	List the contents of a directory
ls -l	The same, with sizes, dates, owners and permissions
cd /path	Change to a directory

Command	What it does
pwd	Print where you currently are
tree	Show a directory and everything under it as a tree

Creating

Command	What it does
mkdir name	Create a directory
mkdir -p a/b/c	Create a directory and any parents it needs
touch file.txt	Create an empty file, or update an existing file's timestamp
cat > file.txt	Create a file and type its contents; press Ctrl+D on its own line to finish

Copying and removing

Command	What it does
cp source dest	Copy a file
cp -r source dest	Copy a directory and everything inside it
mv source dest	Move or rename
rm file	Delete a file — permanently, with no recycle bin
rmdir dir	Delete a directory, which must already be empty and not in use

THERE IS NO RECYCLE BIN

rm deletes. It does not move to a trash folder, it does not ask, and there is no undo. This is the command Vince used on /bin. Before you run rm on anything you did not create yourself, run ls on it first and read what you are about to lose.

TRY THIS

Practise the create-and-verify pattern you will need in PT3:

```
mkdir -p ~/practice && cd ~/practice
cat > note.txt
...type two lines, then press Ctrl+D...
ls -l    &&    cat note.txt
```

10 Checking How Much Space You Have

This is the section Vince skipped, and it is the reason he broke a server. Before you delete anything to free space, measure where the space has actually gone.

```
ubuntu-server: ~
df -h          # space used and free, per mounted file system
du -sh /var/log # how much one directory is actually using
lsblk         # what devices exist and where they are mounted
```

```
sudo fdisk -l          # full partition tables
sudo smartctl -a /dev/sda # drive health from the hardware itself
```

Reading df -h

Column	What it tells you
Filesystem	Which device this line is about
Size	Total capacity of the file system
Used	How much is occupied
Avail	How much is left
Use%	The number you scan for. Anything above 90% needs attention.
Mounted on	Where in the tree it is attached

THE COMMAND THAT WOULD HAVE SAVED VINCE

A single `df -h` would have shown him which file system was at 100%. If the full one had been mounted at `/var`, deleting from `/bin` could not have freed one byte of it — different partition, different pool of space. Measure first. Guessing is what turns a small problem into a rebuild.

11 Windows Partitions, NTFS and Network Shares

11.1 Mounting a Windows partition

On a dual-boot machine, Ubuntu can usually read and write the Windows partition. It often mounts automatically under `/media`; if not, you mount it by hand like any other partition.

```
ubuntu-server: ~
sudo mkdir /mnt/windows
sudo mount /dev/sda2 /mnt/windows
```

THE HIDDEN DANGER IN LINDA'S STORY

Once that partition is mounted, you have write access to Windows system files from Linux — and Windows' own protections do not apply. A careless `rm` in the wrong directory and Windows will not boot. Mount it read-only if you only need to copy files off it.

11.2 When NTFS refuses to mount

If a Windows machine was powered off abruptly, or if Fast Startup or hibernation left the partition in a half-shut-down state, NTFS marks it dirty and Linux will refuse to mount it rather than risk your data. The fix is a package called `ntfs-3g`, the third-generation NTFS driver.

```
ubuntu-server: ~
sudo apt update && sudo apt install ntfs-3g

sudo fdisk -l          # find the partition, listed as HPFS/NTFS/exFAT
```

```
sudo ntfsfix -d /dev/sdXY    # -d clears the dirty flag
```

Replace XY with the real device letters. If ntfsfix cannot repair it, boot into Windows and run chkdsk /f — some NTFS damage can only be repaired by Windows itself.

YOU WILL NEED ntfs-3g IN THE LAB

The same package provides mkfs.ntfs, the tool that creates NTFS file systems. Ubuntu Server does not ship with it, so install it before you try to format the NTFS partition in PT3.

11.3 Network shares

Windows file shares use a protocol called SMB. To reach one from Ubuntu you need CIFS support installed, then you can browse to it or mount it.

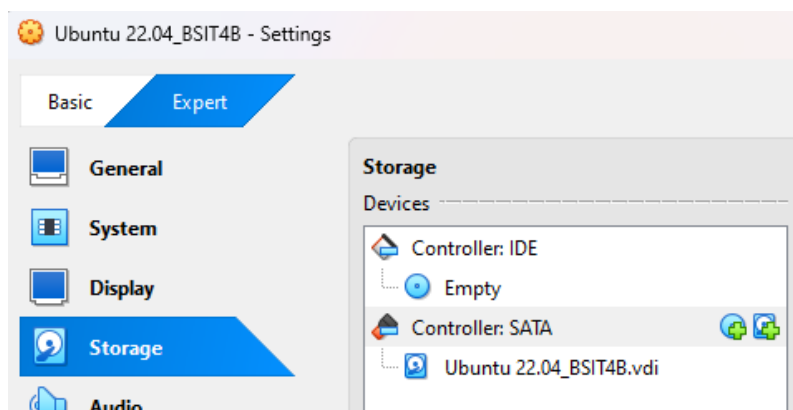
```
ubuntu-server: ~  
sudo apt install cifs-utils  
  
# in a desktop file manager: Go -> Network -> Browse  
# or type the address directly: smb://192.168.1.100/share
```

12 Getting Ready for the Lab: Adding a Virtual Disk

Performance Task 3 begins with two new disks that do not exist yet. Adding a virtual disk in VirtualBox is the equivalent of screwing a physical drive into a server chassis: after it, the hardware is present but the operating system will do nothing with it until you partition and format it.

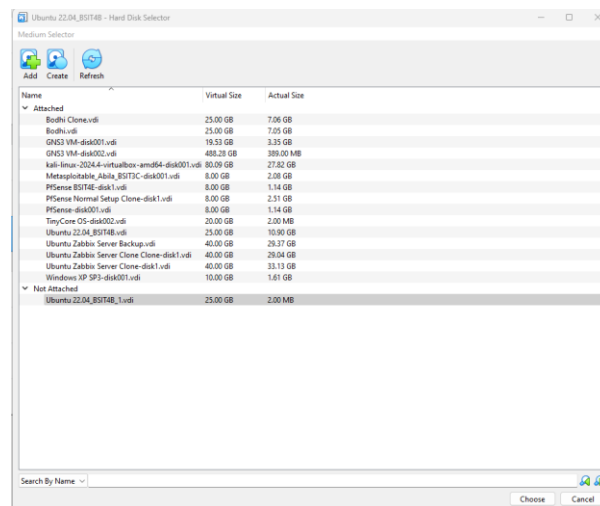
Adding the disk

1. Power the virtual machine off completely — not saved state, powered off.
2. Select the VM in VirtualBox Manager and open Settings, then Storage.
3. Click on Controller: SATA, then the hard-disk icon with the green plus sign.
4. Choose Create, keep VDI, keep Dynamically allocated, then set the size and a filename.
5. Confirm the new disk now appears in the Storage tree under the SATA controller.



The Storage page. New disks attach to the SATA controller.

You can review every virtual disk VirtualBox knows about through File → Virtual Media Manager. This is also where you go to detach or delete one that is no longer needed.



The Virtual Media Manager, listing every virtual disk and its size.

DYNAMIC OR FIXED?

A dynamically allocated disk starts small and grows on the host as you fill it — a 20 GB virtual disk might only occupy 2 GB of real space at first. A fixed disk claims all 20 GB immediately and is slightly faster. For coursework, dynamic is the right choice: it saves host disk space and the performance difference is irrelevant.

SNAPSHOT BEFORE YOU PARTITION

Take a snapshot while the VM is powered off, before you run a single destructive command. It costs thirty seconds and it is the difference between recovering from a mistake and rebuilding your VM from scratch. In PT3 the snapshot is named PT3-START.

13 The Whole Sequence, End to End

Everything in this module, in the order you will actually perform it. Read it through once before you start the lab — recognising the shape of the process is what stops you getting lost halfway.

```
ubuntu-server: ~
# 1. Look at what exists, and find the new disk by its SIZE
lsblk

# 2. Partition it (inside fdisk: g n Enter Enter Enter p w )
sudo fdisk /dev/sdb

# 3. Format the partition
sudo mkfs.ext4 /dev/sdb1

# 4. Create a mount point
sudo mkdir -p /mnt/data

# 5. Mount it
```

```
sudo mount /dev/sdb1 /mnt/data

# 6. Confirm it worked
lsblk -f
df -h

# 7. Use it
sudo cat > /mnt/data/hello.txt

# 8. Detach it cleanly when you are finished
cd /
sudo umount /mnt/data
lsblk
```

ONE THING THAT SURPRISES EVERYONE

After step 8, `ls /mnt/data` shows nothing — the directory is empty. Your file has not been deleted. It is on the partition, and the partition is no longer attached to that directory. Mount it again and the file reappears. Understanding this is understanding mounting.

14 Summary

- Linux uses a single-root hierarchy. Everything hangs beneath `/`, and there are no drive letters.
- Everything is a file, including hardware. Your disks live in `/dev`.
- `ext4` is the default Linux file system. `NTFS` is for sharing with Windows. `FAT32` is for maximum compatibility with small files.
- A disk holds partitions; a partition holds a file system; a file system is reached through a mount point.
- Use `/media` for removable devices the system mounts, and `/mnt` for storage you mount yourself.
- Graphical tools: Disks, GParted, KDE Partition Manager. None of them exist on a server.
- Command-line tools: `lsblk`, `fdisk`, `mkfs`, `mount`, `umount`, `df`.
- File tools: `ls`, `cd`, `mkdir`, `touch`, `cat`, `cp`, `mv`, `rm`, `rmdir`.
- You can mount Windows partitions and network shares, but write access to a Windows system partition is dangerous.
- Always unmount before removing a drive. Always measure with `df -h` before deleting anything to free space.

THE TWO SENTENCES WORTH REMEMBERING

If you cannot say what a directory is for, that is the reason to leave it alone.

Measure before you delete, and unmount before you unplug.

15 Self-Check

Answer these without looking back. The answer key follows on the last page — work through the whole set before you check it, and mark honestly. Anything you get wrong points at a section worth rereading.

Part A — Multiple choice

- Which directory holds the essential system binaries that Vince deleted?
 - /etc
 - /bin
 - /var
 - /tmp
- What is the top of the Linux file system called?
 - The C: drive
 - The home directory
 - The root, written as /
 - The boot partition
- In Linux, are Report.txt and report.txt the same file?
 - Yes, always
 - No, they are two different files
 - Only on ext4
 - Only if they are in different directories
- What does /dev/sdc2 refer to?
 - The second disk, third partition
 - The third disk, second partition
 - A network share
 - A virtual memory file
- Which command shows how much space is free on every mounted file system?
 - ls -l
 - df -h
 - fdisk -l
 - mount
- Which file system can Windows NOT read?
 - FAT32
 - NTFS
 - ext4
 - exFAT
- What must you do to a partition after creating it with fdisk, before it can hold files?
 - Mount it
 - Format it with mkfs
 - Reboot
 - Nothing — it is ready
- Which directory is intended for storage an administrator mounts manually?
 - /media
 - /mnt
 - /home
 - /proc
- What does "target is busy" mean when you run umount?
 - The disk is failing
 - Something is still using the file system
 - The partition is not formatted
 - You need to reboot
- Which key in fdisk writes your changes to the disk?
 - s
 - p
 - w
 - q
- Which package must be installed before you can run mkfs.ntfs on Ubuntu Server?
 - cifs-utils
 - ntfs-3g
 - gparted
 - smartmontools

12. After unmounting a partition, what happens to the files that were on it?

- a. They are deleted
- b. They move to the mount point directory
- c. They stay on the partition and become visible again when remounted
- d. They move to /tmp

Part B — Short answer

Two to four sentences each.

Q1. Linda's thesis was invisible on the Windows lab computer even though the file was intact. Explain why, and say what she should have formatted the USB as instead.

Q2. Explain the difference between partitioning a disk and formatting a partition, and say what would happen if you tried to mount a partition that had been created but never formatted.

Q3. Vince deleted files from /bin to free space on a full root partition. Name the command he should have run first and explain exactly what it would have told him.

Q4. Explain, in your own words, what a mount point is and why it should be an empty directory.

Q5. Why does pulling a USB drive out without unmounting risk corrupting data, when the file appeared to save successfully?

Answer Key

Check your multiple-choice answers first, then read the short-answer notes and compare them with what you wrote. Your wording will differ; what matters is whether you covered the same points.

Part A

#	Ans		#	Ans	
1	b	/bin	7	b	Format it with mkfs
2	c	The root, written as /	8	b	/mnt
3	b	Two different files	9	b	Something is still using it
4	b	Third disk, second partition	10	c	w
5	b	df -h	11	b	ntfs-3g
6	c	ext4	12	c	They stay on the partition

Part B

#	What a good answer covers
1	The USB was formatted ext4, a Linux-native file system that Windows cannot read at all — so the drive appeared blank rather than showing a missing file. She should have used FAT32 for maximum compatibility, or NTFS if she needed files larger than 4 GB.
2	Partitioning reserves a region of the disk and records it in the partition table; it creates a container, not a file system. Formatting writes the file system structures into that container so it can track files. Mounting an unformatted partition fails with an error about a wrong file system type or bad superblock, because there is no structure for the kernel to read.
3	df -h. It reports every mounted file system with its size, how much is used, how much is free and where it is mounted. It would have shown him which file system was actually at capacity. If that was a separate partition such as /var, deleting from /bin could not have freed any space on it — and would have broken the system for nothing.
4	A mount point is an ordinary directory in the single tree that a partition gets attached to, so that reading and writing inside that directory reaches the partition instead of the system disk. It should be empty because mounting hides whatever was already in the directory — the files are not deleted, but they become unreachable until the partition is unmounted, which is confusing and easy to mistake for data loss.
5	Linux buffers writes in memory and flushes them to the disk when convenient, because that is far faster than writing immediately. The command returns and the file looks saved while some of the data is still in memory. Unmounting forces every pending write to complete before detaching. Removing the drive first leaves the file partly written and the file system possibly inconsistent.

Glossary

Term	Meaning
Block device	A storage device the kernel reads and writes in fixed-size blocks. Disks and partitions are block devices; they appear in <code>/dev</code> and in <code>lsblk</code> .
Dirty flag	A marker NTFS sets when a volume was not shut down cleanly. Linux refuses to mount a dirty volume until <code>ntfsfix</code> or Windows <code>chkdsk</code> clears it.
ext4	The default Linux file system. Journaling, full Unix permissions, mature and fast. Unreadable by Windows.
fdisk	The interactive command-line partition editor. Changes are only written when you press <code>w</code> .
File system	The structure written into a partition that tracks which blocks hold which file, plus names, ownership and timestamps.
GPT	GUID Partition Table — the modern partition table format. Supports very large disks and many partitions. Replaces MS-DOS/MBR.
Journaling	A file system feature that records intended changes before making them, so an interrupted write can be recovered rather than leaving corruption.
Mount	To attach a file system to a directory in the tree so its contents become reachable.
Mount point	The directory a file system is attached to. Should be empty before mounting.
NTFS	The Windows file system. Readable and writable from Linux with the <code>ntfs-3g</code> package.
Partition	A logical division of a disk. Each one is independent and can hold a different file system.
Root (/)	The single top of the Linux directory tree. Not the same as the root user, and not the same as <code>/root</code> .
SMART	Health data reported by the drive hardware itself. Read it with <code>smartctl</code> or GNOME Disks.
Snapshot	A saved state of a virtual machine that you can roll back to. Your undo button before destructive work.
Superuser	The administrative account, reached with <code>sudo</code> . Has unrestricted access to every file on the system.
Unmount (<code>umount</code>)	To detach a file system, flushing all pending writes first. Spelled without the <code>n</code> .
UUID	A unique identifier written into a file system when it is created. Unlike device names, it does not change between boots.

WHAT COMES NEXT

Performance Task 3 puts all of this into practice: you will add two virtual disks to your Ubuntu Server, partition one of them into a single ext4 volume and the other into an ext4 partition and an NTFS partition, mount all three, write a file to each, and take them offline cleanly. Read this module before you start it — the lab assumes you already know why each step exists.